SysMonitor

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring. 4 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-SysMonitor-go.

SysMonitor - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/SysMonitor. Entry point: ./cmd/sysmonitor. Module: arrowhead/ah5-support-go/sysmonitor. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8475 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8475"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
SysMonitor	Authorized AMQPS intake, strict observation validation, atomic dedup/history/outbox, subscriptions, history, MQTT QoS1 forwarding	Java MQTTv5 (Go uses 3.1.1), complete live-broker/UI parity, Java history import, broad schema golden tests

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-SysMonitor-rust.

SysMonitor (Rust)

Arrowhead 5.2.2 SysMonitor support system. Entry point src/main.rs is built as binary sysmonitor by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/sysmonitor.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/SysMonitor, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin sysmonitor -- <config.json>

Listening port

Default 127.0.0.1:8475 (Arrowhead default, role sysmonitor). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
SysMonitor	Native mTLS AMQP intake, authorised binding reconciliation, revision subscriptions, SQLite history/outbox, confirmed MQTT output and retention	Full Java DTO/filter parity and wider operational qualification

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-sysMonitor-AMQP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-sysMonitorData-MQTT-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-sysMonitorHistory-HTTP-IDD.tex

- Java: Eclipse-Arrowhead-java/ah5-SysMonitor-java-spring/documentation/v5.2.2/SysMonitor-v5.2.2-sysMonitorManagement-HTTP-IDD.tex

- Go: Eclipse-Arrowhead-go/ah5-SysMonitor-go/documentation/SysMonitor/SysMonitor-HTTPS-IDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/SysMonitor/SysMonitor-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the SysMonitor system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **5**
3.1 struct Observation . . . 5
3.2 Primitives . . . 5

4 References **6**

5 Revision History **7**
5.1 Amendments . . . 7
5.2 Quality Assurance . . . 7

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of SysMonitor.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, SysMonitor registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.SysMonitor.SysMonitor",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `Messaging.Publisher` of this independently packaged SysMonitor system, configured with `AMQP_HOST`, `AMQP_PORT`, `LOCAL_CLOUD` and `MONITOR_INTERVAL_SECONDS` (30); its `echo` value is `status=alive` and the metadata add `implementation=java`. The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 10 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.SysMonitor.SysMonitor.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.SysMonitor.SysMonitor.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.SysMonitor.SysMonitor.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "SysMonitor-echo-1792670400000-42",
  "providerSystem": "SysMonitor",
  "serviceInstance": "SysMonitor",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "SysMonitor is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitorData MQTT/TLS/JSON

Interface Design Description

Service ID: *"sysMonitorData"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `sysMonitorData` service of the SysMonitor support system (SysMonitor-Data-MQTT-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration 3

2 Service Operations **4**
2.1 operation subscribe 4

3 Data Models **5**
3.1 struct StoredObservation 5
3.2 Primitives 5

4 References **6**

5 Revision History **7**
5.1 Amendments 7
5.2 Quality Assurance 7

1 Overview

This document describes the `sysMonitorData` service interface, which provides the stream of all received observations, republished on the local MQTT broker.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	5.0
Data encryption	TLS	1.3 [1], 1.2 [2]
Encoding	JSON	RFC 8259 [3]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitorData` service interface

This document provides the Interface Design Description IDD to the *sysMonitorData – Service Description* document [4].

1.1 Interface registration

```
{
  "serviceDefinitionName": "sysMonitorData",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-Data-MQTT-IDD-5.2.2",
    "schemaVersion": "1.0"
  },
  "interfaces": [
    {
      "templateName": "generic_mqtts",
      "protocol": "ssl",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "mqtt-broker"
        ],
        "accessPort": 8883,
        "baseTopic": "arrowhead/TestCloud/sysmonitor/data",
        "operations": [
          "subscribe"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

2.1 SUBSCRIBE `<baseTopic>/<observationType>/<providerSystem>`

Operation: subscribe
Output: StoredObservation

A consumer connects to the MQTT broker with mutual TLS (its system certificate) and subscribes below the base topic, e.g. `arrowhead/TestCloud/sysmonitor/data/echo/+` or `arrowhead/TestCloud/sysmonitor/data/+/+`. SysMonitor publishes every stored observation with QoS 1, not retained, with `publishedTime` set; the broker's access list decides who may subscribe.

```
{
  "schemaVersion": "string",
  "observationId": "DataManager-echo-1792670400000-42",
  "providerSystem": "DataManager",
  "serviceInstance": "DataManager",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "receivedTime": "2026-10-22T12:00:00.120Z",
  "publishedTime": "2026-10-22T12:00:00.300Z",
  "value": {
    "status": "alive"
  },
  "quality": "good",
  "pipeline": {
    "routingKey": "TestCloud.sysmonitor.DataManager.DataManager.echo"
  }
}
```

Listing 2: A published observation.

3 Data Models

3.1 struct StoredObservation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	Identifier (publisher's, or a SHA-256 of routing key and content).
"providerSystem"	String	Publishing system.
"serviceInstance"	String	Service instance.
"observationType"	String	Observation type.
"sourceTime"	DateTime	Publisher time, UTC.
"receivedTime"	DateTime	Time SysMonitor received it.
"publishedTime"	DateTime	Time it was forwarded on `sysMonitorData`.
"value"	Object	Observed value.
"unit"	String	Unit.
"quality"	String	Quality.
"pipeline"	Object	Intake details (routing key, exchange).

3.2 Primitives

Type	Description
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[2] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[3] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[4] J. Delsing, “sysMonitorData – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorData-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitorHistory HTTP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitorHistory"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitorHistory` service of the SysMonitor system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation list-systems . . . 4
2.2 operation query . . . 4
2.3 operation latest . . . 5

3 Data Models **6**
3.1 struct Latest . . . 6
3.2 struct ObservationList . . . 6
3.3 struct StoredObservation . . . 6
3.4 struct SystemList . . . 6
3.5 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `sysMonitorHistory` service interface of the SysMonitor system, which provides query of the stored sysMonitor observations of every monitored system.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitorHistory` service interface

This document provides the Interface Design Description IDD to the *sysMonitorHistory – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `sysMonitorHistory` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `sysmonitor`, port 8475 and base path `/sysmonitor/history`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`list-systems`	GET	`/systems`
`query`	GET	`/systems/{systemName}/observations`
`latest`	GET	`/systems/{systemName}/latest`

2 Service Operations

The interfaces of the `sysMonitorHistory` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `sysMonitorHistory` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `409 Conflict` (stale revision), `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 GET /sysmonitor/history/systems

Operation: list-systems
Output: SystemList

Returns the systems with stored observations.

A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/history/systems HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An list-systems invocation.

```
{
  "systems": [
    "DataManager",
    "TimeManager"
  ],
  "retentionDays": 30
}
```

Listing 2: An list-systems response.

2.2 GET /sysmonitor/history/systems/*{systemName}*/observations?type=*{type}*&from=*{from}*&to=*{to}*&count=*{count}*

Operation: query
Input: String
Output: ObservationList

Returns observations of a system, optionally by type and period (`count` 1..1000, default 100).

Path parameter: *systemName*.

Query parameters: *type* (List<String>, optional); *from* (String, optional); *to* (String, optional); *count* (Integer, optional, default `100`). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/history/systems/<systemName>/observations?type=<type>&from=<from>&to=<to>&count=<
    count> HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An query invocation.

```
{
  "systemName": "DataManager",
  "count": 1,
  "observations": [
    {
      "schemaVersion": "string",
      "observationId": "DataManager-echo-1792670400000-42",
      "providerSystem": "DataManager",
      "serviceInstance": "DataManager",
      "observationType": "echo",
      "sourceTime": "2026-10-22T12:00:00.000Z",
      "receivedTime": "2026-10-22T12:00:00.120Z",
      "publishedTime": "2026-10-22T12:00:00.300Z",
      "value": {
        "status": "alive"
      },
      "quality": "good",
      "pipeline": {
        "routingKey": "TestCloud.sysmonitor.DataManager.DataManager.echo"
      }
    }
  ]
}
```


Listing 4: An query response.

2.3 GET /sysmonitor/history/systems/*{systemName}*/latest

Operation: latest
Input: String
Output: Latest

Returns the latest observation per type of a system (404 if none).
Path parameter: *systemName*.
A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/history/systems/<systemName>/latest HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An latest invocation.


```
{
  "systemName": "DataManager"
}
```


Listing 6: An latest response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.5, which are used to represent things like hashes, identifiers and texts.

3.1 struct Latest

Object Field	Value Type	Description
"systemName"	String	The system.
"latest"	Object	Latest observation per type.

3.2 struct ObservationList

Object Field	Value Type	Description
"systemName"	String	The system.
"count"	Number	Number of observations.
"observations"	Array<StoredObservation>	Newest first.

3.3 struct StoredObservation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	Identifier (publisher's, or a SHA-256 of routing key and content).
"providerSystem"	String	Publishing system.
"serviceInstance"	String	Service instance.
"observationType"	String	Observation type.
"sourceTime"	DateTime	Publisher time, UTC.
"receivedTime"	DateTime	Time SysMonitor received it.
"publishedTime"	DateTime	Time it was forwarded on `sysMonitorData`.
"value"	Object	Observed value.
"unit"	String	Unit.
"quality"	String	Quality.
"pipeline"	Object	Intake details (routing key, exchange).

3.4 struct SystemList

Object Field	Value Type	Description
"systems"	Array<String>	Systems with stored observations.
"retentionDays"	Object	History retention in days.

3.5 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitorHistory – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorHistory-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitorManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitorManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitorManagement` service of the SysMonitor system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation status . . . 4
2.2 operation providers . . . 4
2.3 operation subscriptions . . . 5
2.4 operation subscribe . . . 5
2.5 operation unsubscribe . . . 6
2.6 operation reconcile . . . 6

3 Data Models **7**
3.1 struct ProviderList . . . 7
3.2 struct ReconcileResult . . . 7
3.3 struct Status . . . 7
3.4 struct Subscription . . . 7
3.5 struct SubscriptionList . . . 7
3.6 struct SubscriptionRequest . . . 8
3.7 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `sysMonitorManagement` service interface of the SysMonitor system, which provides selection of the sysMonitor observations SysMonitor collects: providers found by orchestration, subscriptions per provider and observation type.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitorManagement` service interface

This document provides the Interface Design Description IDD to the *sysMonitorManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `sysMonitorManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `sysmonitor`, port 8475 and base path `/sysmonitor/management`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`status`	GET	`/status`
`providers`	GET	`/providers`
`subscriptions`	GET	`/subscriptions`
`subscribe`	PUT	`/subscriptions/{providerSystem}`
`unsubscribe`	DELETE	`/subscriptions/{providerSystem}`
`reconcile`	POST	`/reconcile`

2 Service Operations

The interfaces of the `sysMonitorManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `sysMonitorManagement` SD document, as well as input and output types.

Every request needs mutual TLS with a system or operator certificate of the cloud and `Authorization: Bearer IDENTITY-TOKEN//<token>`; the token is verified with Authentication and must belong to the system named by the certificate. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with `{"error": "<text>"}` and the status codes `400 Bad Request` (invalid input), `401 Unauthorized` (no valid identity), `403 Forbidden` (caller not permitted), `404 Not Found`, `409 Conflict` (stale revision), `500 Internal Server Error`, `502 Bad Gateway` (a system it depends on failed) and `503 Service Unavailable` (overloaded).

2.1 GET /sysmonitor/management/status

Operation: status
Output: Status

Returns the SysMonitor status.

A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/management/status HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An status invocation.

```
{
  "status": "ok",
  "system": "Gatekeeper",
  "implementation": "java",
  "version": "5.2.2",
  "subscriptions": 1
}
```

Listing 2: An status response.

2.2 GET /sysmonitor/management/providers

Operation: providers
Output: ProviderList

Returns the orchestrated sysMonitor providers.

A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/management/providers HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An providers invocation.

```
{
  "defaultSubscription": "all",
  "list": [
    {}
  ]
```

```
}
```

Listing 4: An providers response.

2.3 GET /sysmonitor/management/subscriptions

Operation: subscriptions
Output: SubscriptionList

Returns the subscriptions.
A successful call returns `200 OK` with the output in the body.

```
GET /sysmonitor/management/subscriptions HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An subscriptions invocation.

```
{
  "defaultSubscription": "all",
  "list": [
    {
      "providerSystem": "DataManager",
      "observationTypes": [
        "*"
      ],
      "enabled": true,
      "revision": 3,
      "updatedAt": "2026-10-22T12:00:00Z",
      "updatedBy": "Sysop"
    }
  ]
}
```

Listing 6: An subscriptions response.

2.4 PUT /sysmonitor/management/subscriptions/*{providerSystem}*

Operation: subscribe
Input: SubscriptionRequest
Output: Subscription

Creates or updates the subscription for a provider and applies it to the broker bindings.
Path parameter: *providerSystem*.
Optional `If-Match: <revision>`; a stale revision gives 409.
A successful call returns `200 OK` with the output in the body.

```
PUT /sysmonitor/management/subscriptions/<providerSystem> HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "observationTypes": [
    "*"
  ],
  "enabled": true
}
```

Listing 7: An subscribe invocation.

```
{
  "providerSystem": "DataManager",
  "observationTypes": [
    "*"
  ],
  "enabled": true,
  "revision": 3,
  "updatedAt": "2026-10-22T12:00:00Z",
  "updatedBy": "Sysop"
}
```

Listing 8: An subscribe response.

2.5 DELETE /sysmonitor/management/subscriptions/*{providerSystem}*

Operation: **unsubscribe**
Input: **String**

Removes the subscription for a provider (204).
Path parameter: *providerSystem*.
A successful call returns `204 No Content`.

```
DELETE /sysmonitor/management/subscriptions/<providerSystem> HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 9: An unsubscribe invocation.

2.6 POST /sysmonitor/management/reconcile

Operation: **reconcile**
Output: **ReconcileResult**

Re-orchestrates the providers now (202).
A successful call returns `202 Accepted` with the output in the body.

```
POST /sysmonitor/management/reconcile HTTP/1.1
Host: sysmonitor:8475
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 10: An reconcile invocation.

```
{
  "jobId": "reconcile-1792670400"
}
```

Listing 11: An reconcile response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ProviderList

Object Field	Value Type	Description
"defaultSubscription"	String	`all` or `none` (`SYSMONITOR_DEFAULT_SUBSCRIPTION`).
"list"	Array<Object>	Orchestrated sysMonitor providers: `providerSystem`, `serviceInstanceId`, `baseRoutingKey`, `exchange`, `observationTypes`, `broker`, `port`, `orchestratedAt`.

3.2 struct ReconcileResult

Object Field	Value Type	Description
"jobId"	String	Identifier of the started re-orchestration.

3.3 struct Status

Object Field	Value Type	Description
"status"	String	`running` or `degraded`.
"system"	String	`SysMonitor`.
"implementation"	String	`java`.
"version"	Version	`5.2.2`.
"amqp"	Object	Intake connection, bindings and last message time.
"storage"	Object	History counts, retention and pruning.
"subscriptions"	Number	Number of subscriptions.

3.4 struct Subscription

Object Field	Value Type	Description
"providerSystem"	String	Provider system.
"observationTypes"	Array<String>	Selected types.
"enabled"	Boolean	Whether active.
"revision"	Number	Revision, also returned as `ETag`.
"updatedAt"	DateTime	Time of the last change.
"updatedBy"	String	Last editor.

3.5 struct SubscriptionList

Object Field	Value Type	Description
"defaultSubscription"	String	`all` or `none`.
"list"	Array<Subscription>	Subscriptions with `providerFound` and `unknownTypes`.

3.6 struct SubscriptionRequest

Object Field	Value Type	Description
"observationTypes"	Array<String>	Types to receive; `*` for all (default).
"enabled"	Boolean	Default true.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.
Version	JSON String, semantic version `major.minor.patch`.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitorManagement – Service Description (SysMonitor),” Eclipse Arrowhead, SD 5.2.2, 2026, sysMonitor-v5.2.2-sysMonitorManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

HTTPS, mandatory mutual TLS, Arrowhead identity-token header and JSON request/response documents. General management is sysop/whitelist protected; enrollment endpoints use approved certificate fingerprints and profile/serial checks instead of ordinary identity tokens. Error responses contain errorMessage, errorCode, origin and timestamp. Exact DTO and status/error parity is not certified.

1.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
GET	/sysmonitor/management/status	management
GET	/sysmonitor/management/providers	management
GET	/sysmonitor/management/subscriptions	management
PUT	/sysmonitor/management/subscriptions/{providerSystem}	management
DELETE	/sysmonitor/management/subscriptions/{providerSystem}	management
POST	/sysmonitor/management/reconcile	management
GET	/sysmonitor/history/systems	identity
GET	/sysmonitor/history/systems/{systemName}/latest	identity
GET	/sysmonitor/history/systems/{systemName}/observations	identity
GET	/sysmonitor/monitor/echo	certificate
GET	/sysmonitor/general/mgmt/get-config	management
POST	/sysmonitor/general/mgmt/logs	management

1.2 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- AMQPS intake and MQTT QoS1 outbox forwarding are implemented. MQTT uses Paho 3.1.1 rather than Java MQTTv5. Full live-broker/UI parity and imported Java history have not been certified. The default subscription is none until configured.
- The source reference uses a shared non-Spring HTTP dispatcher. Its route coverage is not inferred from Go route presence; manual JavaSupportRuntime review is required.

1 SysMonitor

1.1 System overview

Native Rust development profile for local Java source `JavaSupportRuntime/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Revision subscriptions, strict diagnostic intake, atomic history/outbox, queries

1.5 Open release gates

AMQP/MQTT workers and orchestration reconciliation absent; status degraded; no automated retention; history cap100000. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `echo`, `generalManagement`, `sysMonitorHistory`, `sysMonitorManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/sysmonitor/general/mgmt/get-config	management
POST	/sysmonitor/general/mgmt/logs	management
GET	/sysmonitor/monitor/echo	certificate
GET	/sysmonitor/management/status	management
GET	/sysmonitor/management/providers	management
GET	/sysmonitor/management/subscriptions	management
PUT	/sysmonitor/management/subscriptions/{providerSystem}	management
DELETE	/sysmonitor/management/subscriptions/{providerSystem}	management
POST	/sysmonitor/management/reconcile	management / 501
GET	/sysmonitor/history/systems	identity
GET	/sysmonitor/history/systems/{systemName}/latest	identity
GET	/sysmonitor/history/systems/{systemName}/observations	identity
POST	/sysmonitor/observations	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.